



תרגיל בית מס. 2 נושאים מרכזיים:

Files, Generics, Collections

הוראות הגשה: בזוגות, עד לתאריך 02/06/26.

- שם הפרוייקט: "ProjectRescue2".
- כל המחלקות צריכות להיות בחבילה (package) בשם: "rescue".
- יש להגיש קובץ zip שמכיל בתוכו את כל הקבצים .java. הרלוונטיים. נא וודאו פעם נוספת שכל הקבצים הנדרשים נמצאים. במידה ועניתם על שאלת הבונוס: נא לכלול בתוך ה-zip גם קובץ .txt. עם התשובה לשאלה.
- השם של הקובץ יהיה בפורמט הבא:
ID1_ID2_HW2.zip ○
- בקשות הארכה מסיבות מוצדקות - חייבות להתבצע לפני מועד ההגשה. נתון לשיקול דעתם של המרצה/מתרגל.
- מותר להשתמש רק במבני הנתונים שנלמדו עד כה במסגרת הקורס, כגון מערך רגיל או רשימה.

מבנה ההגשה:

ID1_ID2_HW2.zip

```
|
|
|--- src/
|   |--- rescue/
|       |--- Main.java
|       |--- Identifiable.java
|       |--- Registry.java
|       |--- FileManager.java
|   |--- שאר מחלקות המערכת
|
|--- data/
|   |--- קבצי הבדיקה
|
|--- output/
|   |--- קבצי הפלט
```

```
|
└── screenshots/
    ├── run_basic.png
    ├── run_advanced.png
    └── report_created.png
```

התרגיל ייבדק אוטומטית. לכן יש להקפיד על שמות מחלקות, שמות מתודות, שמות קבצים, סדר שורות, פורמט פלט, רווחים, פסיקים ואותיות גדולות/קטנות.

התוכנית תרוץ כך:

```
java rescue.Main data/commands_basic.txt
```

במקרה של שגיאת ארגומנטים יש להדפיס בדיוק אחת מההודעות הבאות:

ERROR: Missing commands file

ERROR: Invalid arguments

ERROR: Cannot read commands file

בזמן ריצה תקינה אין להדפיס למסך דבר.

התרגיל מתבסס על הפתרון שלכם מתרגיל בית 1. כל המחלקות, הממשקים, ה Enums וההחריגות מתרגיל בית 1 ממשיכים להתקיים ולעבוד. מותר להוסיף להם שדות ומתודות לפי הצורך.

בתרגיל בית 2 יש להוסיף רק את המחלקות/הממשקים החדשים הבאים:

Identifiable

Registry

FileManager

NationalRescueAuthority תמשיך להיות המחלקה המרכזית שמנהלת את מצב המערכת.

Identifiable

יש להוסיף ממשק גנרי בשם Identifiable, הממשק מייצג אובייקט שיש לו מפתח ייחודי.

הגדרה:

```
public interface Identifiable<K> {
    K getKey();
}
```

Responder חייב לממש:

Identifiable<Integer>

ב Responder המתודה getKey מחזירה את id

Incident חייב לממש :

Identifiable<String>

ב Incident-המתודה getKey מחזירה מפתח בפורמט : TYPE-SERIAL

לדוגמה : FIRE-1

Registry

יש להוסיף מחלקה גנרית בשם Registry, המחלקה מנהלת אובייקטים לפי מפתח.
כותרת המחלקה :

```
public class Registry<K, T extends Identifiable<K>>
```

שדה חובה :

```
private HashMap<K, T> items;
```

הבנאי :

```
public Registry()
```

הבנאי יאתחל HashMap ריק.

מתודות חובה :

```
public boolean add(T item)
```

המתודה מוסיפה אובייקט חדש.

```
public boolean removeByKey(K key)
```

המתודה מסירה אובייקט לפי מפתח.

```
public T get(K key)
```

המתודה מחזירה אובייקט לפי מפתח.

אם לא קיים אובייקט עם מפתח כזה, יש להחזיר null

```
public boolean containsKey(K key)
```

המתודה מחזירה true אם קיים אובייקט עם המפתח הנתון

```
public int size()
```

המתודה מחזירה את מספר האובייקטים השמורים.

```
public ArrayList<T> getAll()
```

המתודה מחזירה את כל האובייקטים בתוך ArrayList חדש.

public void clear()
המתודה מוחקת את כל האובייקטים.

FileManager

יש להוסיף מחלקה בשם FileManager

המחלקה אחראית על קריאת קבצים, ביצוע פקודות מקובץ וכתיבת שגיאות.
אין שדות חובה במחלקה זו.

מתודות חובה:

public static void loadResponders(String path, NationalRescueAuthority system)
המתודה קוראת קובץ משתתפים, יוצרת Medic, Dispatcher או Drone לפי הנתונים, ומוסיפה את
המשתתפים התקינים למערכת.

public static void loadIncidents(String path, NationalRescueAuthority system)
המתודה קוראת קובץ אירועים, יוצרת אובייקטי Incident, ומוסיפה את האירועים התקינים למערכת.

public static void processCommands(String path, NationalRescueAuthority system)
המתודה קוראת קובץ פקודות ומבצעת את הפקודות לפי הסדר שבו הן מופיעות בקובץ.

public static void logError(String fileName, int lineNumber, String message)
המתודה מוסיפה שורת שגיאה לקובץ:
output/errors.log
פורמט שגיאה:
ERROR,fileName,lineNumber,message

public static void clearErrorLog()
המתודה מנקה את הקובץ output/errors.log בתחילת כל ריצה. אם הקובץ לא קיים, יש ליצור אותו.

עדכונים ב NationalRescueAuthority

יש לעדכן את NationalRescueAuthority כך שתנהל את המשתתפים והאירועים בעזרת Registry

שדות חובה:

private Registry<Integer, Responder> responders;
private Registry<String, Incident> incidents;

בנוסף, יש לשמור את ערכי המערכת שהוגדרו בתרגיל בית 1, כגון מגבלת שעות עבודה, שכר מינימלי, וערך
ברירת מחדל לשעות עבודה של רחפן ללא טעינה.

חובה להשתמש לפחות בשלושה אוספים שונים מתוך Java Collections Framework במהלך הפתרון, למשל
LinkedList, HashMap, ArrayList, TreeSet, PriorityQueue, HashSet

מתודות חובה:

```
public boolean addResponder(Responder r)  
מוסיפה משתתף למערכת.
```

```
public boolean removeResponder(Responder r)  
מסירה משתתף לפי id
```

```
public Responder getResponderById(int id)  
מחזירה משתתף לפי id
```

```
public ArrayList<Responder> getAllResponders()  
מחזירה רשימה של כל המשתתפים במערכת.
```

```
public boolean addIncident(Incident in)  
מוסיפה אירוע למערכת.
```

```
public boolean removeIncident(Incident in)  
מסירה אירוע לפי המפתח שלו.
```

```
public Incident getIncidentByKey(String key)  
מחזירה אירוע לפי מפתח.
```

```
public ArrayList<Incident> getAllIncidents()  
מחזירה רשימה של כל האירועים במערכת.
```

```
public ArrayList<Responder> getRespondersByType(String type)  
מחזירה את כל המשתתפים מהסוג המבוקש.  
הערכים החוקיים הם:
```

MEDIC

DISPATCHER

DRONE

סדר המיון:

MEDIC — לפי id בסדר עולה.

DISPATCHER — לפי experience בסדר עולה, ובשוויון לפי id

DRONE — לפי workNoCharge בסדר עולה, ובשוויון לפי id

אם type אינו תקין, יש להחזיר רשימה ריקה.

```
public ArrayList<Incident> getAllIncidentsSorted()
```

מחזירה את כל האירועים ממוינים לפי TYPE בסדר אלפביתי עולה, ובשוויון לפי SERIAL בסדר עולה.

```
public void assignAll()
```

מנסה להקצות משתתפים לכל האירועים במערכת לפי כללי ההקצאה מתרגיל בית 1.

האירועים יטופלו לפי TYPE בסדר אלפביתי עולה, ואז SERIAL בסדר עולה.

אירוע שכבר הוקצה לא יוקצה שוב.

אירוע שלא ניתן להקצות יישאר לא מוקצה.

```
public void exportReport(String path)
```

יוצרת קובץ דוח מלא לפי הפורמט שמוגדר בהמשך.

אם לא ניתן לכתוב את הקובץ, יש לרשום שגיאה ל errors.log-בפורמט :

```
ERROR,path,0,Cannot write file
```

קובץ המשתתפים

קובץ המשתתפים הוא קובץ CSV השורה הראשונה היא שורת כותרת ויש לדלג עליה.

מבנה השורה :

TYPE,ID,NAME,CLEARANCE,SPECIALIZATION,EXPERIENCE,DRINK,WORK_NO_CHARGE,FAILURES,SALARY

כל שורה חייבת להכיל בדיוק 10 שדות.

TYPE – סוג המשתתף.

ערכים חוקיים :

MEDIC

DISPATCHER

DRONE

ID – מזהה המשתתף.

יש לבדוק לפי הכללים מתרגיל בית 1.

NAME – שם המשתתף.

רלוונטי ל MEDIC ול DISPATCHER

עבור DRONE השדה צריך להיות ריק.

CLEARANCE – רמת הסיווג.

ערכים חוקיים :

LOW

MEDIUM

HIGH

MEDIC – רלוונטי ל-SPECIALIZATION
הערכים החוקיים הם הערכים שהוגדרו בenum Specialization
עבור DISPATCHER ו-DRONE השדה צריך להיות ריק.

DISPATCHER – רלוונטי ל-EXPERIENCE
הערך הוא מספר שלם.
עבור MEDIC ו-DRONE השדה צריך להיות ריק.

DISPATCHER – רלוונטי ל-DRINK
הערכים החוקיים הם הערכים שהוגדרו ב-
עבור MEDIC ו-DRONE השדה צריך להיות ריק.

DRONE – רלוונטי ל-WORK_NO_CHARGE
הערך הוא מספר ממשי.
עבור MEDIC ו-DISPATCHER השדה צריך להיות ריק.

DRONE – רלוונטי ל-FAILURES
הערך הוא מספר שלם.
עבור MEDIC ו-DISPATCHER השדה צריך להיות ריק.

DISPATCHER ול-MEDIC – רלוונטי ל-SALARY
יש לבדוק לפי כללי השכר מתרגיל בית 1.
עבור DRONE השדה צריך להיות ריק.

אם שורה אינה תקינה, אין לעצור את התוכנית. יש לדלג על השורה ולרשום שגיאה ל-output/errors.log

קובץ האירועים

קובץ האירועים הוא קובץ CSV השורה הראשונה היא שורת כותרת ויש לדלג עליה.

מבנה השורה:

TYPE,SERIAL,CLEARANCE,HOURS,DISPATCHERS,ROOM,MEDICS,DRONES

כל שורה חייבת להכיל בדיוק 8 שדות.

TYPE – סוג האירוע.
הערך הוא מחרוזת שאינה ריקה.

SERIAL – מספר סידורי של האירוע.
הערך חייב להיות מספר שלם חיובי.

CLEARANCE – רמת הסיווג הנדרשת.

ערכים חוקיים :

LOW

MEDIUM

HIGH

HOURS – מספר שעות העבודה הדרוש לאירוע.

הערך חייב להיות מספר ממשי חיובי.

DISPATCHERS – מספר המוקדנים הדרוש.

הערך חייב להיות מספר שלם שאינו שלילי.

ROOM – מספר חדר הבקרה.

הערך חייב להיות מספר שלם חיובי.

MEDICS – מספר הפרמדיקים הדרוש.

הערך חייב להיות מספר שלם שאינו שלילי.

DRONES – מספר הרחפנים הדרוש.

הערך חייב להיות מספר שלם שאינו שלילי.

המפתח של אירוע הוא :

TYPE-SERIAL

אם כבר קיים אירוע עם אותו מפתח, יש לרשום שגיאה.

אם שורה אינה תקינה, אין לעצור את התוכנית. יש לדלג על השורה ולרשום שגיאה ל־output/errors.log

הודעות שגיאה בקבצי קלט

כל שגיאה תיכתב לקובץ :

output/errors.log

פורמט :

ERROR,fileName,lineNumber,message

lineNumber הוא מספר השורה בקובץ, כולל שורת הכותרת.

הודעות שגיאה אפשריות :

Invalid row format

Invalid type

Invalid id

Invalid clearance

Invalid specialization
Invalid experience
Invalid drink
Invalid work no charge
Invalid failures
Invalid salary
Duplicate responder
Invalid incident type
Invalid serial
Invalid hours
Invalid dispatchers
Invalid room
Invalid medics
Invalid drones
Duplicate incident
Cannot read file
Cannot write file

אם אין שגיאות, הקובץ errors.log צריך להיות קובץ ריק.

קובץ הפקודות

התוכנית מקבלת כארגומנט נתיב לקובץ פקודות.
יש לדלג על שורות ריקות ועל שורות שמתחילות ב#. המערכת תומכת בשלוש פקודות בלבד:
LOAD_DATA respondersPath incidentsPath

הפקודה קוראת את קובץ המשתתפים ואת קובץ האירועים ומוסיפה את הנתונים התקינים למערכת.
אם אחד הקבצים לא ניתן לקריאה, יש לרשום:
ERROR,path,0,Cannot read file

ASSIGN_ALL

הפקודה מנסה להקצות משתתפים לכל האירועים במערכת לפי כללי תרגיל בית 1.
אם אירוע לא ניתן להקצאה, יש לרשום שגיאה בפורמט:
ERROR,assign_all,incidentKey,message

incidentKey הוא:
TYPE-SERIAL

הודעות אפשריות:
Not enough dispatchers

Not enough medics

Not enough drones

EXPORT_REPORT path

הפקודה יוצרת דוח מלא של מצב המערכת בקובץ path.

אם מופיעה פקודה לא מוכרת, יש לרשום :

ERROR,commands,lineNumber,Unknown command

אם מספר הפרמטרים של פקודה אינו תקין, יש לרשום :

ERROR,commands,lineNumber,Invalid command format

קובץ הדוח

הדוח נוצר על ידי הפקודה :

EXPORT_REPORT path

כל ערך double ייכתב עם ספרה אחת אחרי הנקודה.

מבנה הדוח :

PROJECT_RESCUE_REPORT

RESPONDERS_COUNT=<number>

INCIDENTS_COUNT=<number>

ASSIGNED_INCIDENTS_COUNT=<number>

UNASSIGNED_INCIDENTS_COUNT=<number>

RESPONDERS

<responder line>

<responder line>

INCIDENTS

<incident line>

<incident line>

END_REPORT

RESPONDERS_COUNT – מספר כל המשתתפים במערכת.

INCIDENTS_COUNT – מספר כל האירועים במערכת.

ASSIGNED_INCIDENTS_COUNT – מספר האירועים שהוקצו בהצלחה.

UNASSIGNED_INCIDENTS_COUNT – מספר האירועים שלא הוקצו.

המשתתפים יודפסו לפי הסדר :

MEDIC

DISPATCHER

DRONE

בתוך כל סוג יש להשתמש בסדר שהוגדר במתודה `getRespondersByType`

מבנה שורת MEDIC

MEDIC,id,name,clearance,specialization,workHours,salary,busy

מבנה שורת DISPATCHER

DISPATCHER,id,name,clearance,experience,drink,workHours,salary,busy

מבנה שורת DRONE

DRONE,id,clearance,workNoCharge,reportedFailures,busy

האירועים יודפסו לפי TYPE בסדר אלפביתי עולה, ובשוויון לפי SERIAL בסדר עולה.

מבנה שורת INCIDENT

INCIDENT,type,serial,clearance,hours,status,contributorsCount

status יהיה : ASSIGNED או UNASSIGNED

אם האירוע לא הוקצה `contributorsCount` יהיה 0.

אין להוסיף רווחים לפני או אחרי פסיקים.

אין להוסיף רווחים לפני או אחרי הסימן =

אין להוסיף שורות או טקסט שלא מופיעים בפורמט הנדרש.

Main

המחלקה Main תכיל את המתודה :

`public static void main(String[] args)`

אם `args.length == 0` יש להדפיס :

ERROR: Missing commands file

אם `args.length > 1` יש להדפיס :

ERROR: Invalid arguments

אם קובץ הפקודות אינו קיים או אינו ניתן לקריאה, יש להדפיס :

ERROR: Cannot read commands file

במקרה תקין Main, תבצע :

ניקוי `errors.log`

קבצי בדיקה וצילומי מסך

ייתנו קבצי בדיקה לדוגמה בתיקיית data וקבצי expected_output להשוואה.

הבדיקה האוטומטית עשויה להשתמש בקבצים אחרים באותו מבנה.

יש לצרף תיקיית screenshots ובה שלושה צילומי מסך :

run_basic.png

run_advanced.png

report_created.png

run_basic.png - הרצה עם commands_basic.txt

run_advanced.png - הרצה עם commands_advanced.txt

report_created.png - צילום שמראה שקבצי הפלט נוצרו בתיקיית output

בהצלחה!